Valid Vault Enterprise Security Design

Nov 12, 2026 · Michael Carter

1. Verdict and scope

The design is defensible on two points. Every key that protects a vault resists guessing far beyond practical attack, and phase 1 runs with no password-management server, database, or login page for anyone to attack.

The personal vault protections in this document are in the shipping code (0.9.4.7). Everything marked planned is the phase 1 enterprise design and has not been built or tested. A compromised unlocked device, a malicious update, theft of encrypted files, and loss of the file share stay real risks, and sections 10 and 11 cover them.

2. Threat model

These are the attacks the design is built to stop, and what it promises against each.

Threat	Security objective	Status
Theft of files from the shared folder	No passwords or usable keys recovered, and no password-derived data to guess against	Planned
An employee reaching vaults not assigned to them	Each device opens only the shared vaults whose keys were wrapped to it	Planned
A departed employee	No access to anything saved after removal	Planned
A member posing as an admin	Settings and membership change only with the company signature, plus executive signatures where the company requires them	Planned
A lost admin device	Admin access restored through the separate recovery split	Planned
Tampering with shared files	Altered, damaged, or replayed files rejected before use	Planned

Threat	Security objective	Status
A stolen key backup file	Guessing costs every passphrase times every answer, at 1,000,000 rounds a guess	Shipping
A copied personal vault	Each guess costs 600,000 rounds of PBKDF2, with no stored hash to test against	Shipping

What the design does not promise:

- An unlocked device with malware on it stays exposed. Malware reads whatever that person can open, in any password manager.
- Anything an employee already saw or copied stays with them. Passwords they could open must be changed after they leave.
- Whoever controls the file share can delete or withhold files. Devices detect missing or stale data and refuse to treat it as current, and backups restore it.
- A malicious update from a hijacked store account or IT deployment runs with the extension's full access.
- Physical access to an unlocked device is out of scope.

3. Key protection and encryption

Every vault is encrypted with AES-256-GCM, and every encryption binds a context label as authenticated data so a ciphertext cannot be moved to another purpose. Attackers go after whatever unlocks the key, so each unlock method is protected separately. No standalone password hash is stored anywhere. A device proves it holds the right secret only by unwrapping the key.

Shipping in the personal vault:

Unlock method	Protection	What a thief with the stored data faces
Master password	PBKDF2-SHA256, 600,000 rounds, 12 characters minimum	About 15,000 guesses a second on one high-end GPU (estimate). A random 12-character password takes billions of years

Unlock method	Protection	What a thief with the stored data faces
Fingerprint, browser	Key wrapped under a WebAuthn PRF output bound to the device's authenticator	Nothing to guess off the device. See the legacy gap in section 11
Android unlock	Key held in the Android hardware keystore, released after fingerprint or screen lock	Nothing to guess off the device
Encrypted key backup file	Passphrase plus three Compound Questions, joined into one secret, PBKDF2-SHA256 at 1,000,000 rounds	Every combination at once, with one yes or no for all four parts (section 4)

Planned for enterprise phase 1:

Mechanism	Protection
Device keys	A P-256 ECDH key pair and a P-256 ECDSA key pair per device, private halves stored wrapped under the device's master key
Shared vault keys	Random 256-bit AES-GCM keys, wrapped to each member device with ephemeral ECDH, HKDF-SHA256, and AES-GCM
Company authority	A P-256 ECDSA signing key held by the admin
Executive approval	Each executive's own P-256 ECDSA key, with a company-set threshold
Recovery	A P-256 ECDH recovery key pair. Vault keys wrap to its public key, and its private key is split k of n among trusted people

4. Compound Questions and the key backup file

The key backup file is protected by a compound secret: the owner's passphrase and three answers. All four are joined into one input before key derivation, and nothing checks any part on its own. A guess either derives the right key or it does not, and a wrong guess

never says which part was wrong.

So the attacker's search space is the product of the passphrase candidates and the plausible values for each answer. Each answer multiplies the work.

What the attacker must guess	Combinations	Time at about 8,800 guesses a second
A passphrase narrowed to 1 million candidates	1 million	About 2 minutes
That passphrase, plus three answers with about 1,000 plausible values each	1 quadrillion	About 3,600 years
A random 16-character passphrase, before any answers	About 10^31	Far longer than the age of the universe

Assumptions behind the table:

- The passphrase must be at least 16 characters with a letter, a number, and a symbol, and is screened against common passwords. The owner picks three of 10 Compound Questions, and answers are case-sensitive.
- 8,800 guesses a second is an estimate for one current high-end GPU at 1,000,000 rounds, scaled from published PBKDF2-SHA256 rates. Stage 1 of the build plan replaces it with a measured rate (hashcat mode 10900).
- 1,000 plausible values per answer is illustrative. Someone who knows the owner may narrow an answer much further, so against a close attacker the passphrase is the real floor. The key file is never weaker than its passphrase alone.
- Checking the parts separately would be weaker: each check would give its own yes or no, and the guesses would add instead of multiply.

5. Enterprise architecture (planned)

Part	Where it lives	What it holds or does
Member device	Each employee's browser extension or Android app	The personal vault, plus the device's key pairs, wrapped under its master key. Contacts no Valid Vault service

Part	Where it lives	What it holds or does
Shared vault folder	The company's file server or NAS, reached over the office network or VPN	The signed member list (manifest), vault keys wrapped to device public keys and the recovery key, and encrypted, signed change files
Company signing key	The admin's desktop, wrapped under the admin's master key	Signs company settings and every membership change
Company settings	Pushed through Chrome policy and Android managed configuration	Company rules and public keys, signed by the company key
Recovery key	Split into pieces, such as 3 of 5, held offline by trusted people	Restores admin access and reopens shared vaults after a lost admin device
Phones	Employee phones	Receive shared vaults from a member desktop by QR code. Read and fill only
Updates	Chrome Web Store, or the company's own software deployment	New versions of the extension and app

How the parts work together:

1. The admin creates the company. Valid Vault makes the company signing key and the recovery key, and the recovery key is split and handed out.
2. Each employee device makes its own key pairs and shows a short code built from its public keys.
3. The admin matches the code, signs a new manifest that adds the device, and wraps each assigned vault key to the device's public key.
4. Devices write encrypted, signed change files into their own folder inside the shared vault.
5. Every device checks the manifest's company signature and each change file's device signature, then unwraps its own copy of the vault key and merges the changes.

A drawn diagram of these key and file flows is planned for this section.

6. Membership, authorization, and device trust (planned)

First trust. A device's first settings import pins the company's public key after the admin and employee compare a short code. Every later settings file or manifest must verify against that pinned key.

Signed settings and membership. Settings and manifests carry an epoch number that only goes up. A device remembers the highest epoch it has accepted and refuses anything lower, or anything at the same epoch with different contents. This blocks rollback to an older member list.

Device signatures. Each change file is signed by the device that wrote it. A signature shows which enrolled device wrote a change. It does not prove which person was at the keyboard.

Authorization rules. Signatures only matter if devices enforce who may sign what:

Action	Who may do it	How devices verify it
Add, edit, or delete entries in a shared vault	Any member device with write access in the manifest	Device signature, and the device listed as a writer at that epoch
Add or remove members	The admin	Company signature on the new manifest
Change company settings	The admin	Company signature on the settings file
Protected actions the company lists, such as ownership change or admin recovery	The admin plus the required executives	Company signature plus the threshold of executive signatures
Anything administrative inside a change file	Nobody	Change files can only carry vault entries. Any other content is rejected

7. Revocation, re-keying, and offline devices (planned)

When an employee leaves, the admin removes their devices. Valid Vault then re-keys every shared vault those devices could open:

1. Writes an encrypted journal on the admin device recording the old and new key generations.
2. Makes a new random vault key and wraps it to each remaining device and the recovery key.
3. Writes a full encrypted snapshot of the vault under the new key.
4. Reads the snapshot back and compares a fingerprint of its full contents against the journal.
5. Signs and publishes the new manifest at the next epoch.
6. Retires the old generation's files after the retention period the company sets.

If the process stops at any step, the journal lets the next run finish it or roll it back. A removed device never receives the new key, so it cannot read anything saved afterward.

What re-keying cannot do: erase what a removed device already holds. A local-copy device keeps the snapshot it had, still locked by its own unlock, until it is wiped. When it reconnects, it finds itself missing from the manifest and refuses to sync. Network-only devices lose access as soon as they are removed. Deleting old files from the share removes the company's copies, not copies a device already made.

8. Recovery and multi-executive approval (planned)

Recovery. The recovery key protects against losing the admin device. It is split so that any k of n pieces, such as 3 of 5, rebuild it, and fewer reveal nothing. The split is written in-house, as decided. Building it to the SLIP-0039 format is proposed, so its published test vectors apply directly. Because vault keys wrap to the recovery public key, re-keys never need the pieces. A rebuilt key is checked by deriving its public key and comparing it to the one in the signed settings, with no stored hash, and it is cleared from memory once the new admin device is enrolled.

Executive approval. Separately, the company lists actions that need several executives to sign, and sets the threshold. Each executive signs with their own device key. Devices refuse a protected action that lacks the required signatures.

The two do different jobs. Recovery gets the company back in after a loss. Executive approval stops one admin from acting alone.

9. Storage, endpoint, and deployment security

- **BitLocker:** required on company laptops through the deployment checklist, so a stolen drive cannot be copied off for offline guessing.
- **Network-only shared vaults:** no shared vault copy is written to the laptop. Windows Offline Files must be turned off for that share, or Windows may cache the files locally.
- **Local copies:** the shared snapshot is stored encrypted, locked by the device's own unlock.
- **Wipe after failed unlocks:** erases the local vault and device keys after the set number of wrong tries at the unlock screen. It does nothing against offline guessing on a copied drive, which BitLocker covers.
- **File share permissions and backups:** only enrolled users get write access to the folder, and the company's normal backups cover it.
- **Extension network policy:** the extension's content security policy sets connect-src to none, which blocks network requests from extension pages. File access through Chrome's File System Access API is operating-system file access and is not governed by that policy. Reaching a mapped drive still needs the office network or VPN.
- **Updates:** come from the Chrome Web Store or the company's own deployment. That channel is trusted, as with every extension.
- **Android:** phones get shared vaults from a member desktop by QR and use the hardware keystore. Managed configuration delivers company settings.

10. Threat scenarios and limits

Scenario	Protection	What remains	Still needed
Stolen employee laptop	BitLocker, network-only option, device removal and re-key	A local copy stays, locked by the master password, until wiped	Change the passwords that laptop could open
Compromised unlocked device	Exposure limited to what that device can open	Malware reads it all while unlocked	Endpoint security, outside Valid Vault
Malicious admin	Executive approval on the actions the company protects	Unprotected admin actions	Each company chooses its protected list at setup

Scenario	Protection	What remains	Still needed
Stolen company signing key	Executive approval on protected actions	Forged settings and member changes until the key is replaced	A key rotation procedure, built in stage 5
Compromised update channel	Store and IT deployment controls	A bad version runs with full access	Same for every extension
Files deleted or withheld	Devices keep their copies and spot missing data	Shared work stops until restored	Company backups
Rollback or replay of an old manifest	Epoch numbers, refused when lower or duplicated	None known	Tests in stage 3
Two people save at once	Each device writes only its own change files, merged by edit counter	Same-entry edits resolve to one winner	Merge rule settled before stage 4
Interrupted re-key	Encrypted journal, finish or roll back, then verify	None known	Tests in stage 5
Recovery pieces stolen	Below the threshold they reveal nothing	At the threshold, full admin recovery	Pieces kept apart, offline

11. Implementation validation and known gaps

Known gap, shipping code. Installs from before hardware-bound fingerprint unlock stored a copy of the key locked with a fixed secret in the code. Anyone who copies that install's data can open its vault without guessing. The copy is deleted only when the person next unlocks by fingerprint. The fix: delete that copy on any successful unlock, password included, and ask the person to set up fingerprint again. It affects only installs upgraded from that old version and not unlocked by fingerprint since.

Tests that must pass before the enterprise claims stand:

Test	Build stage
Shared-folder reads, writes, listing, and deletion on Chrome, Edge, and Brave over a mapped drive and a NAS	0
ECDH wrapping and HKDF context binding with fixed test vectors, plus public key validation	1
Signature verification over canonical bytes, with tampered inputs rejected	1
Recovery split against the published SLIP-0039 vectors, including wrong and short piece sets	1
Measured guess rate for the key backup file	1
Manifest rollback and replay rejected	3
Member devices cannot sign admin actions	3
AES-GCM nonces never repeat across concurrent writers under one key	4
Concurrent edits merge the same on every device	4
Thousands of damaged and altered files rejected without a crash	4
Interrupted re-key at every step finishes or rolls back	5
A removed device reads nothing saved after removal	5
Full deployment on a real NAS, network-only and local-copy	7
Outside security review	8

12. Claims and comparison

Use the claims on the left. Each one holds up when a security person checks it, once its tests pass.

Say this	Not this
No password-management server or database is required	Zero attack surface, or unhackable
Shared vaults are encrypted files on company-controlled storage	Nothing to attack in the shared folder
No standalone password hash is stored, so every guess pays the full key derivation	Uncrackable
A stolen key backup with a narrowed passphrase and three private answers takes thousands of years to guess	Trillions of years, for every key file
The extension contacts no Valid Vault service	The extension makes no network connections
Each change is signed by the device that wrote it	The admin can see which person made each change
Re-keying keeps removed devices out of everything saved afterward	Removal erases what a device already had
Sensitive actions can require signatures from several executives	Built and shipping today
Fill-only devices are planned for phase 2	A filled password cannot be read out of the page

Compared with self-hosted Bitwarden: Bitwarden runs self-hosted and on isolated networks, and its enterprise product has far more mature administration, reporting, SSO, and integrations. The difference in phase 1 is the server tier.

Exposure	Self-hosted Bitwarden	Valid Vault phase 1
A service accepting connections	Web vault, API, and identity services over HTTPS	None
A login page to attack	Yes	None
A central database	Every user's encrypted vault in one SQL database	None. Encrypted files on the existing share

Exposure	Self-hosted Bitwarden	Valid Vault phase 1
Server software to patch	Server containers and their host	None
Stolen data that can be attacked by guessing passwords	Each user's key, protected by their master password. Researchers cracked real server databases offline (ivision)	Vault keys wrapped to device public keys and the recovery public key, with no password-derived wraps

One Bitwarden claim from earlier research is still unchecked: that its offline mode is read-only. Verify it before repeating it.

Sources

- Bitwarden: Encryption Key Derivation
- Bitwarden: License Organizations or Premium (self-hosted)
- ivision Research: Bitwarden server hashcat plugin, conclusion
- SLIP-0039: Shamir's Secret-Sharing for Mnemonic Codes
- Valid Vault Enterprise Phase 1 Build Plan